Advanced Data Management
Mock Exam v3 (Chapters 1–6): Full Solutions

Every answer in the rubric's shape: the letter or the number first, the formula, the computation with units, the one-sentence decision, the trap

Advanced Data Management

2026-11-01 100 points 2 hours

How to use this document

Mock v3 keeps Mock v2's structure (8 quick checks, 3 situations, 1 real plan, 2 of 3 tickets) on a new database, `citycab`, so that nothing can be answered from memory of v2. Every solution is written the way the rubric grades: letter or number first, formula, computation with units, the one-sentence decision, then the trap answer and why it is wrong. All three tickets are answered.

Part A: Quick checks

A1. **(b) the binder, the catalog.** The parser only checks the grammar, and `drvier_id` is a perfectly grammatical identifier; the binder resolves every name against the catalog and finds no such column ("column drvier_id does not exist"). The planner never sees the query.

A2. **(c) 120.** Record = 40+24 = 64 bytes (already a multiple of 8), plus the 4-byte slot = 68; $\lfloor 8{,}168/68 \rfloor = \lfloor 120.1 \rfloor = 120$. Check: 2,000,000/120 = 16,667 pages, the table's size. (a) forgets header and slot (8,168/40), (b) forgets the slot (8,192/64).

A3. **(b) 4.** Levels = $\lceil \log_{200} 2{,}000{,}000 \rceil$: $200^2 = 40{,}000 < 2{,}000{,}000 \leq 200^3 = 8{,}000{,}000$, so 3 levels; one lookup reads 3 index pages plus the heap page = 4. (c)/(d) use $\log_2$ ($2^{21} \approx 2$M): a binary tree, not a B^+-tree.

A4. **(b) a B-tree on `lower(email)`.** An index is sorted on its key expression; `lower(email)` is a different key from `email`, so (a) and (c) cannot be used at all (the planner would have to compute `lower` on every row). BRIN (d) only helps a physically ordered column and never a point lookup on a function.

A5. **(c) HashAggregate.** A group's count is final only after the last input row, so nothing can be emitted before the input is exhausted: a pipeline

breaker. A filtered Seq Scan streams every surviving row; Nested Loop and Merge Join emit a pair as soon as it matches (Merge Join needs sorted *inputs*, but the sorting is a separate node).

A6. **(c) 66,668.** Runs $= \lceil 16{,}667/130 \rceil = 129$; fan-in $M-1 = 129 \geq 129$, so one merge pass; total passes $= 1+1 = 2$; I/O $= 2 \times 16{,}667 \times 2 = 66{,}668$. (a) counts one read, (b) one pass read and written, (d) assumes two merge passes.

A7. **(a) about 6,000.** Both values are in their MCV lists: $0.01 \times 0.30 = 0.003$ under independence, $0.003 \times 2{,}000{,}000 = 6{,}000$. (b) is `status` alone, (c) `zone_id` alone, (d) their sum, the classic AND/OR confusion.

A8. **(b) the count becomes 2 and the hand moves on.** Clock-sweep decrements `usage_count` (0–5) as the hand passes and evicts only a frame it finds at 0; a hit increments the count (up to 5) outside the hand's visit. Pinned frames are skipped entirely; (d) confuses the pin count with the usage count.

Part B: Problem situations

Solution: B1 "An index on a 2 % filter never helps, delete it"

(a) $s = 7/365 = 0.0192$ (1.92 %); rows $= 0.0192 \times 2{,}000{,}000 = 38{,}356$; contiguous, they fill $\lceil 38{,}356/120 \rceil = 320$ pages.
(b) Seq Scan $= 16{,}667 \times 1 + 2{,}000{,}000 \times 0.01 + 2{,}000{,}000 \times 0.0025 = 16{,}667 + 20{,}000 + 5{,}000 = 41{,}667$.
Index Scan, correlation 0.99: descent $3 \times 4 = 12$; leaves $\lceil 38{,}356/300 \rceil = 128$ pages, chained and read in order; heap: the 320 contiguous pages, read in order (320, or 1,280 if charged at the random price); CPU $38{,}356 \times (0.005 + 0.01) = 575$. Total $\approx 12 + 128 + 320 + 575 \approx 1{,}035$ (at worst $\approx 2{,}000$). The index wins by a factor of about 40 (20 at worst).
(c) Correlation 0.003: the 38,356 rows are scattered; expected heap pages $= 16{,}667 \times (1-(1-0.0192)^{120})$; $(0.9808)^{120} = e^{120 \ln 0.9808} = e^{-2.32} = 0.098$, so $0.902 \times 16{,}667 \approx 15{,}035$ pages, 90 % of the table, each a random read: $15{,}035 \times 4 = 60{,}140$, plus $12 + 128 + 575$: about $60{,}855 > 41{,}667$. The Seq Scan wins (a bitmap scan would read the same 15,035 pages once, in order, about 15,000 plus CPU, roughly a draw). The rule: $1/(4r) = 1/480 = 0.21$ %, and 1.92 % is nine times above it, so for an *unordered* table the colleague is right.
Decision: keep the index. The rule $s < 1/(4r)$ assumes matching rows scattered one per page; with correlation 0.99 the week's rides sit in 320 consecutive pages and the index scan costs about 1,000 units against 41,667. The single word is **correlation** (pages, not rows). A BRIN index on `ride_ts` would give the same win in a few pages of space. **Trap:** applying $1/(4r)$ without looking at `pg_stats.correlation`.

Solution: B2 Driver earnings, sorted by driver

(a) Block nested loop, `drivers` outer: $B_R + \lceil B_R/(M-2) \rceil \times B_S = 160 + \lceil 160/62 \rceil \times 16{,}667 = 160 + 3 \times 16{,}667 = 50{,}161$ page I/Os. `rides` is read three times.
(b) Build side = the smaller input, `drivers`, 160 pages $> M = 64$: it does not fit, so the Grace (partitioned) hash join applies: both inputs are partitioned on `driver_id` into $M-1$ partitions, written, then joined pair by pair: $3(B_R + B_S) = 3 \times 16{,}827 = 50{,}481$. Slightly *more* than the block nested loop here, because three chunks of the outer cost the same as three reads of S while Grace reads and writes both inputs.
(c) Sort `rides`: runs $= \lceil 16{,}667/64 \rceil = 261$; merge passes $= \lceil \log_{63} 261 \rceil = 2$ ($63 < 261 \leq 3{,}969$); I/O $= 2 \times 16{,}667 \times 3 = 100{,}002$. Sort `drivers`: runs $= \lceil 160/64 \rceil = 3$, one merge pass, I/O $= 2 \times 160 \times 2 = 640$. Merge: $160 + 16{,}667 = 16{,}827$. Total $= 100{,}002 + 640 + 16{,}827 = 117{,}469$, more than twice the other two.
(d) $M = 200$. Block NL: $\lceil 160/198 \rceil = 1$ chunk: $160 + 16{,}667 = 16{,}827$. Hash: $160 \leq 200$, build fits: 16,827, one read of each table. Sort-merge: `rides` runs $\lceil 16{,}667/200 \rceil = 84$, one merge pass, 66,668; `drivers` fits, 320; merge 16,827; total 83,815. **Decision:** run the hash join with `drivers` as the build side (`work_mem` $\geq$ 160 pages = 1.3 MB; the 4 MB default already holds it, so $M = 64$ was an artificial restriction). The tie with the block nested loop is only in I/O: in memory the nested loop compares every `rides` row with every driver of the chunk, $2{,}000{,}000 \times 20{,}000$ comparisons, while the hash join does one probe per row. For the sorted output, do not sort the 16,667-page input: aggregate first (HashAggregate on 20,000 drivers, a few hundred KB) and sort the 20,000-row result, a single in-memory quicksort. **Trap:** sorting the big input to satisfy `ORDER BY`; and using $|R| = 20{,}000$ rows in the nested-loop formula (that is the simple nested loop, $160 + 20{,}000 \times 16{,}667$, absurd).

Solution: B3 "The morning after the audit"

(a) EAT $= 0.995 \times 0.1 + 0.005 \times 100 = 0.0995 + 0.5 = 0.60\,\mu$s: six times RAM; the 0.5 % of misses already cost five times more than all the hits together. At 90 %: $0.09 + 10 = 10.09\,\mu$s, 17× slower than the daytime value and 100× RAM.
(b) Zero pages survive. The scan streams 400,000 use-once pages through 131,072 frames; after the first 131,072 scan pages every frame holds a scan page, and each hot page's last use is older than every scan page's, so LRU evicts all 20,000 of them (the scan is three times longer than needed to do it). In the morning every hot page returns by a miss: $20{,}000 \times 100\,\mu$s = 2 s of pure reload, spread over the first transactions as a collapsed hit ratio (the "slow morning").
(c) Threshold = `shared_buffers`/4 = 32,768 pages (256 MB); $400{,}000 >$ 32,768, so the scan uses a ring of 32 frames (256 KB) and recycles them among itself (page k evicts page $k-32$). The working set survives because $20{,}000 \leq 131{,}072 - 32$. The ring costs the scan nothing: every one of its

pages is a compulsory miss and would not have been hit again anyway. So on PostgreSQL the slow morning has another cause, for instance the OS page cache (which the ring does not protect) or a bulk `COPY` ring of 16 MB; measure with `pg_buffercache` after the job.
(d) The scan gains nothing from 48 GB: a use-once read of 3.2 GB has hit ratio ≈ 0 at any pool size, and the ring keeps it out of the pool anyway. Meanwhile the OS cache shrinks to under 16 GB minus `work_mem` per connection, and checkpoints must flush up to 48 GB of dirty pages in bursts. Before deciding, measure the working set in `pg_buffercache` (20,000 pages = 160 MB: the 1 GB pool is already six times larger) and the hit ratio from `pg_statio_user_tables`; the number says the pool is not the problem. **Trap:** "more cache is always better"; and reading "99.5 %" as "nearly perfect" without the EAT.

Part C: Read a real plan

Solution: Ticket OPS-44, questions 1 to 5

1. The tree. Limit ← Sort ← HashAggregate ← Nested Loop with two children: outer = Bitmap Heap Scan on `rides` (whose own child is the Bitmap Index Scan), inner = Index Scan on `riders`, run once per outer row.

Node	All input first?	Why
Bitmap Index Scan	**yes**	builds the complete bitmap of matching row ids before the heap scan can start
Bitmap Heap Scan	no	walks the bitmap page by page and emits rows as it goes
Index Scan riders	no	one lookup per outer row, emits the match immediately
Nested Loop	no	emits each (ride, rider) pair as soon as the inner returns
HashAggregate	**yes**	a rider's count is final only after the last ride
Sort	**yes**	the largest count is known only after all groups are in
Limit	no	passes the first 20 rows it receives, then stops

The `LIMIT` cannot help because two breakers sit below it: the Sort cannot hand over its first row before the HashAggregate has finished, which cannot finish before the whole join has run. "Stop after 20" only saves the sort's output phase (`top-N heapsort`, 27 kB). **Trap:** calling the Bitmap Heap Scan blocking (it streams) or the Bitmap Index Scan streaming (it does not).
2. Estimates. `rows=40000`: `zone_id = 7` is in the MCV list with frequency 0.02, so $0.02 \times 2{,}000{,}000 = 40{,}000$, and it is exact (actual 40,000). `rows=4000`: the Filter `pickup_city = 'Rabat'` has MCV frequency 0.10, and the planner multiplies under independence: $40{,}000 \times 0.10 = 4{,}000$. Ac-

tual 40,000, `Rows Removed by Filter: 0`: every zone-7 ride is in Rabat, because a zone lies inside one city. q-error = 40,000/4,000 = 10, an under-estimate, the dangerous direction. Failure 2, correlated columns: `zone_id` determines `pickup_city`. `ANALYZE rides` does not fix it: both frequencies are already fresh and correct; the error is the multiplication. Fix: `CREATE STATISTICS s_geo (dependencies) ON zone_id, pickup_city FROM rides; ANALYZE rides;` after which the planner knows `pickup_city` is implied by `zone_id` and estimates $\approx$ 40,000.

3. Buffers. `Heap Blocks: exact=15190`: 40,000 rows are 2 % of the table, but with correlation 0.003 they are spread over the file; the expected number of pages with at least one match is $16{,}667 \times (1 - 0.98^{120}) = 16{,}667 \times (1 - 0.089) = 15{,}190$, 91 % of `rides` (about 2.6 matches per page). The bitmap reads each of them once, in physical order, all from the pool (`hit=15193`, warm). Inner Index Scan: `loops=40000`; each lookup descends the 3-level `riders_pkey` and fetches one heap page, 4 pages per loop, $40{,}000 \times 4 = 160{,}000 = 121{,}400 + 38{,}600$. The index pages (root, inner level, often the leaf) are hot and hit; the heap pages of `riders` are 160 MB, more than the pool, and 31,207 distinct riders means 38,600 of those heap fetches miss. The two seconds are those misses: $38{,}600 \times 50\,\mu\text{s} \approx 1.9$ s, matching `actual time` 1,890 ms on the Nested Loop against 310 ms for the whole bitmap scan. The overall hit ratio 136,593/175,193 = 78 % is not useful: the cost is in the 38,600 `read`s, random, and a ratio hides it. **Trap:** "read means disk"; here some of the 38,600 come from the OS cache, which is why it is 50 and not 100 μs each.

4. The right plan. The nested loop was chosen because 4,000 lookups at 4.07 each (16,280) looked cheaper than hashing `riders`. With 40,000 outer rows the lookups cost $40{,}000 \times 4.07 = 162{,}800$ units (the actual 1.9 s). Hash join alternative: build on the *smaller* side, which is now the filtered `rides`: 40,000 rows $\times$ about 70 bytes = 2.8 MB $<$ 4 MB `work_mem`, `Batches: 1`; probe with a Seq Scan of `riders`: $20{,}000 + 2{,}000{,}000 \times 0.01 = 40{,}000$ units. Total $\approx 17{,}833 + 40{,}000 + 0.0025 \times 2{,}040{,}000 \approx 63{,}000$ units, 2.6 times less, and the real time is better still: 20,000 *sequential* pages (160 MB at 500 MB/s $\approx$ 0.3 s) against 38,600 random reads. Expected total about 0.6 s instead of 1.95 s. The join order flips: `rides` becomes the build (inner) side under the Hash node, `riders` the probe. **Trap:** building the hash on `riders` (20,000 pages, 8 batches).

5. The proposals. (i) Composite index `(zone_id, pickup_city)`: the `Filter` line becomes part of the `Index Cond`, `Rows Removed by Filter` stays 0 (it already was), `Heap Blocks` stays 15,190 (same 40,000 rows on the same pages), the estimate stays 4,000 (the index condition is costed with the same independence rule), and the inner loop still runs 40,000 times: nothing changes except a slower insert. (ii) `CLUSTER rides USING rides_zone_idx`: rewrites the table in `zone_id` order, correlation $\to 1$; the 40,000 rows now sit in $\lceil 40{,}000/120 \rceil = 334$ consecutive pages, so `Heap Blocks` drops from 15,190 to about 334 and the bitmap scan from 310 ms to a few ms; the estimate is unchanged and the 40,000 random rider

lookups remain, so the dashboard goes from 1.95 s to about 1.65 s. CLUSTER is also a one-off 130 MB rewrite under an exclusive lock, and new rides append out of order. **Decision:** first the extended statistics (two commands, no rewrite): they fix the join and the time drops to about 0.6 s; then, if zone dashboards are the main workload, consider CLUSTER or a partial index per hot zone. **Trap:** fixing the scan when the join is the cost.

Part D: Discussion

Solution: Ticket D1 "Move rides to a column store"

Claim. Column storage helps the analytics scan by roughly a factor of eight and hurts every OLTP operation by a factor of 25; the answer is both, not one. **Mechanism and numbers.** The analytics query touches 3 of 25 columns. In a row store it must read every page, 16,667 (130 MB), because the three columns live inside every row. In a column store each column is a separate file, so it reads $3/25 = 12\,\%$ of the bytes, about 2,000 page-equivalents, and column files of similar values compress well (dates and zone ids by run-length or delta), often another factor of 3 to 5: 16,667 pages become a few hundred. For the app: an insert of one ride must append to 25 column files instead of writing one slotted-page tuple, 25 writes instead of 1 (and 25 `fsync`s under WAL), and a point read of a ride by `id` reassembles the row from 25 places. At 2,000 inserts per second, 50,000 column writes per second against 2,000. **Consequence and recommendation.** Keep `rides` as a row store for the app (OLTP: short transactions, whole rows by key, B-trees). Give analytics a column copy: export nightly with `COPY (SELECT zone_id, ride_ts, fare FROM rides WHERE ride_ts >= current_date - 1) TO '/data/rides.parquet' (FORMAT parquet)` and query it with DuckDB (`read_parquet`), which reads only the referenced column chunks and row groups. Inside PostgreSQL, two cheaper steps first: a BRIN index on `ride_ts` (the table is append-ordered) so that per-day reports read only that day's block ranges, and a covering index `ON rides (ride_ts) INCLUDE (zone_id, fare)` for an index-only scan (`Heap Fetches: 0`). **Trap:** "column stores are simply better": worse for single-row reads and writes by the number of columns.

Solution: Ticket D2 "Replace clock-sweep with true LRU"

Claim. LRU is not optimal, exact LRU is too expensive to be worth its small gain, and the real enemy of every recency policy is the sequential scan. **Mechanism.** The optimal policy is Belady's OPT: evict the page whose next use is furthest in the future; it needs the future, so it is only a yardstick. LRU approximates it by assuming the past predicts the future (recently used $\Rightarrow$ soon used again). The stack property (more frames never cause more misses, unlike FIFO's Belady anomaly) is a nice guarantee,

not optimality. **Cost.** Exact LRU keeps a doubly linked list ordered by last use; *every hit* unlinks and relinks the frame, under a lock shared by all backends. At a 99.5 % hit ratio, hits are 99.5 % of all page accesses, millions per second on a busy server: the list becomes a global serialization point and the latency of a hit (0.1 μs of memory) is dominated by the lock. **Approximation.** Clock keeps one reference bit per frame and a hand: a hit sets the bit without any lock on a shared structure; the hand clears bits and evicts the first frame with bit 0, which is "not used since the hand last passed", a coarse LRU. PostgreSQL's clock-sweep uses a `usage_count` saturating at 5 instead of one bit: a page needs several hand passes without a hit to be evicted, which adds *frequency* to recency (a page hit five times survives a page hit once), cheaply. **The failing workload.** A sequential scan larger than the pool: every page is used once, so OPT, LRU, clock and FIFO all fill the pool with pages that will never be reused and evict the hot working set (sequential flooding). PostgreSQL does not fix this with the policy but with intent: a scan of a table larger than `shared_buffers`/4 runs through a 32-frame ring, `COPY` and `VACUUM` through rings of their own, so the scan recycles its own frames and the hot set survives. **Consequence.** Replacing clock-sweep with a list would make hits slower for a marginal gain in hit ratio and would not touch the one case that actually hurts. **Trap:** "LRU is optimal".

Solution: Ticket D3 "Set join_collapse_limit = 1 everywhere"

Claim. Freezing the written join order server-wide trades 30 seconds of planning once for bad plans on every query; fix the one query instead. **Numbers.** Nine tables have 9! = 362,880 left-deep join orders (more with bushy trees), each combinable with three join algorithms and several access paths. System-R dynamic programming does not enumerate them: it keeps the cheapest plan for every non-empty subset of tables, $2^9 - 1 = 511$ subsets, built level by level, plus one extra plan per *interesting order* (a sort order a later merge join or the `ORDER BY` can reuse). That is why 30 seconds is unusual even for nine tables: the cost is usually in interesting orders and in costing many index paths, and beyond `join_collapse_limit` (8 by default) PostgreSQL already stops collapsing explicit `JOIN`s, and beyond `geqo_threshold` (12) it switches to genetic search. **What the setting does.** `join_collapse_limit = 1` tells the planner to join exactly in the order the `FROM` clause lists the tables; it still chooses algorithms and access paths, but not the order. **Why the written order is rarely the best.** The best order joins the most selective inputs first so that intermediate results stay small: selection pushdown shrinks each table to its filtered rows, and the planner wants the smallest filtered input as the build or outer side of the first join. A developer writes tables in the order of the business question, not of their filtered sizes; if the 2,000,000-row table with no filter comes first, the first join produces millions of rows that every later join must process, and a join that should produce 40,000 rows produces them after 10^9 intermediate ones. A frozen

order is also wrong next month, when the data distribution changes. **Instead.** (1) Find out why planning is slow: `EXPLAIN` the query, count the indexes and the interesting orders; (2) if the query is a long report, 30 s of planning against minutes of execution is acceptable, and a prepared statement or `plan_cache_mode` plans it once; (3) if the written order is known to be good, set `SET LOCAL join_collapse_limit = 1` inside that one transaction, or restructure that query with CTEs or a materialised intermediate table; (4) leave the default for the rest of the server. **Trap:** 9! as "the number of plans the planner evaluates" (it evaluates 511 subsets).

Summary: Where each question lives in the textbooks

- A1: Chapter 1 (life of a query, the binder). A2: Chapter 2 (slotted page, rows per page). A3, A4: Chapter 3 (B^+-tree height, expression indexes). A5, A6: Chapter 4 (pipeline breakers, external sort). A7: Chapter 5 (MCV, independence). A8: Chapter 6 (clock-sweep).
- B1: Chapter 3 §4–5 and Chapter 5 §5 (index versus scan, correlation). B2: Chapter 4 §4–5 (join costs, memory). B3: Chapter 6 §1, §5 (EAT, flooding, rings, sizing).
- C: Chapter 4 §7 (reading a whole plan), Chapter 5 §4 and §7 (independence, the three failures, access path), Chapter 3 §4 (pages touched, CLUSTER), Chapter 6 §7 (the Buffers line).
- D1: Chapters 1–2 (OLTP versus OLAP, row versus column). D2: Chapter 6 §4–5 (policies, rings). D3: Chapter 5 §2 and §6 (pushdown, System-R DP, the levers).